

Projeto Prático #1: O projeto deve ser desenvolvido por grupos de *no máximo* três alunos. É estritamente proibida a partilha de código entre alunos de grupos diferentes.

Plágio: Além de inspeção manual, será também executado um software de deteção de plágio no código-fonte do projeto. *Todos* os alunos que submeterem código obtido através de plágio terão os seus projetos anulados.

Uso de Inteligência Artificial: É permitido utilizar ferramentas de IA como o ChatGPT, Copilot, DeepSeek e similares. No entanto, todos os membros do grupo devem ser capazes de compreender e *explicar* o projeto *na sua totalidade* aos docentes quando pedido. Se houver código no projeto que o grupo não consiga explicar, o projeto será considerado como *plágio*.

Submissão: A data de submissão do projeto é **10/04/2026** às 23:59. A descrição do processo de submissão pode ser encontrada no enunciado.

TascaEats: Delivery de Comida

1 Contextualização

A consultoria espanhola **Compadres Sempre Saciados (CSS¹)** identificou uma oportunidade no mercado de entregas ao domicílio em Portugal e decidiu criar o **TascaEats**, um sistema de delivery de comida. O objetivo é desenvolver uma plataforma robusta que conecte clientes, restaurantes e entregadores.

Um utilizador pode interagir com o sistema com diferentes perfis: como Cliente, como Administrador do Restaurante ou como Entregador. Todos partilham dados base de autenticação e identificação, mas têm permissões e papéis distintos². Para já, um cliente terá só uma morada mas no futuro poderá ter várias. O sistema deve permitir o registo, a atualização e a remoção de utilizadores, bem como a autenticação (que nesta fase será feita via mock).

Para a gestão de restaurantes, o sistema permite a um administrador criar e gerir um ou mais restaurantes, definindo o nome, o NIF e a morada do estabelecimento. O restaurante possui um cardápio composto por vários produtos, cada um com nome, descrição, preço e estado de disponibilidade. O restaurante pode também abrir e fechar para receber pedidos.

No que diz respeito ao processo de compra, o cliente pode selecionar produtos de um restaurante e criar um pedido. A cada pedido associa-se a data e hora, o valor total, o endereço de entrega e o estado atual (e.g., Criado, Pago, ...). É obrigatório que cada pedido tenha um pagamento associado, que pode ser de diferentes tipos (e.g., Multibanco, MBWay, Dinheiro, ...).

Por fim, o sistema deve gerir a logística. Quando um pedido está pronto, é alocado a um entregador disponível. O sistema deve manter o registo das entregas, associando o entregador, o pedido e os horários de retirada e entrega no cliente final.

Nesta **Fase 1**, o foco será o desenvolvimento do *backend* como um sistema monolítico, expondo a lógica de negócio exclusivamente através de uma API REST³.

¹Entidade fictícia

²Por simplificação, um utilizador não pode ter múltiplos papéis simultaneamente

³A descrição dos atributos e entidades acima **não é suficiente para solução completa**

2 Casos de Uso

Numa versão inicial, pretende-se implementar alguns casos de uso fundamentais, deixando a gestão avançada de estados e a interface gráfica para uma segunda fase. Apresenta-se a lista completa dos casos de uso para que os alunos tenham a perceção do domínio total. Para a primeira fase, devem implementar os casos de uso identificados abaixo. Mais casos de uso podem ser adicionados nas próximas fases.

- A. Login com autenticação (Nota: vamos fazer mock. Qualquer palavra-passe será aceite con-
tanto que o utilizador seja válido).
- B. Registo de utilizadores (Clientes, Administradores e Entregadores).
- C. Verificar, remover e atualizar utilizadores.
- D. Buscar por restaurantes (por nome ou cidade).
- E. Criação de restaurantes e gestão do seu estado (aberto/fechado).
- F. Adicionar, remover e atualizar produtos no cardápio de um restaurante.
- G. Buscar produtos de um restaurante.
- H. Criação de um pedido.
 - I. Registo e processamento do pagamento associado a um pedido.
 - J. Atualização do estado do pedido.
- K. Atribuição de um entregador a um pedido
- L. Cancelamento de um pedido

3 Requisitos Não Funcionais

O projecto tem ainda os seguintes requisitos:

- A autenticação poderá ser feita via mock (futura integração com sistema real).
- Toda a informação deverá ser armazenada numa base de dados relacional.
- A plataforma deverá ser implementada em Spring Boot, usando Java. A camada de dados será implementada utilizando JPA/Spring Data.
- A aplicação deverá lidar com pedidos concorrentes, sem criar inconsistências.
- A camada de negócios deverá usar um Domain Model rico e fazer o uso correto de anotações JPA (incluindo herança e composição onde aplicável) para facilitar o mapeamento.
- A implementação da solução deve utilizar o conceitos visto em aula (e.g., camadas, DTOs, controllers, testes, etc.) e seguir as boas práticas de desenvolvimento de software.
- A interface será através de *endpoints REST*. Deve ser possível aceder diretamente a esses end-
points através do Swagger (<https://swagger.io/>). Não haverá desenvolvimento de interface
gráfica (HTML/GUI) nesta fase.

- **Submissão de vídeos demonstrativos da execução dos casos de uso** (detalhes na secção 4).
- O repositório deve aceitar apenas código com o nível de qualidade aceite pela equipa (ex.: *pre-commit*, testes).
- O projecto correrá sempre num ambiente Docker.
- O controlo da participação de cada membro do grupo será feito via atividade no repositório git, com base no número e qualidade dos *commits*.

4 Tarefas da Fase 1

A equipa deverá:

- Fazer fork do repositório original (<https://git.alunos.di.fc.ul.pt/css000/tascaeats>), para a conta de um dos elementos do grupo.
- Dar acesso a todos os membros do grupo, como Maintainer.
- Dar acesso à conta css000, como Reporter.
- Alterar o `README.md` para identificar a equipa e **incluir os links para o(s) vídeo(s) demonstrativo(s)**.
- Implementar os *endpoints REST* necessários (com acesso através do Swagger) para os casos de uso implementados.
- Cada grupo deve gravar um pequeno vídeo demonstrando os seus casos de uso a funcionar. A demonstração deve ser feita consumindo a API REST (e.g., via Swagger), mostrando o envio do JSON de requisição e a resposta do servidor/alteração na base de dados. Os vídeos podem ser alojados na nuvem (e.g., Google Drive com permissão de leitura, YouTube) com os links inseridos no `README.md`; ou adicionados diretamente ao repositório do projeto.

Nesta etapa não deverá ser entregue a aplicação completa com interface gráfica.

4.1 Validação

Devem ter atenção a garantir que as regras de negócio da aplicações estão implementadas e demonstradas nos vídeos. Alguns exemplos de regras de negócio necessários:

- Um pedido não pode ser cancelado se já estiver em preparo
- Um produto associado a um pedido antigo não pode ser completamente removido do cardápio (soft-delete)
- Não é possível fazer pedido a restaurantes fechados
- Não devem permitir adicionar um produto ao pedido se estiver esgotado
- Não devem associar uma entrega a um entregador que não está disponível

Este não é um conjunto exaustivo das regras de negócio, serve somente para ilustrar casos que o sistema deve garantir e isso deve ser demonstrado nos vídeos.

Para validar o funcionamento da aplicação, também devem garantir que o modelo de base de dados gerado deve ser capaz de responder a questões de negócio como estas:

- Quais os restaurantes que possuem o maior volume de vendas em euros?
- Qual é a morada do restaurante com mais vendas? E do cliente com mais pedidos?
- Em média, quantos pedidos um cliente realiza por mês?
- Quais os produtos mais vendidos da plataforma?
- Qual é o método de pagamento mais utilizado pelos clientes?
- Quais os clientes que se registaram mas ainda não realizaram nenhuma compra?

5 Como Entregar

Para entregar o trabalho, basta criar uma tag chamada *fase1* e enviá-la para o repositório. O repositório deverá conter o código-fonte do projecto, o ficheiro `docker-compose.yml` necessário para correr a aplicação e a base de dados. Não se esqueça de incluir os links dos vídeos no ficheiro `README.md`.

```
git tag fase1
git push origin fase1
```

Deve-se confirmar que o vosso projecto está acessível à conta CSS000 na tag *fase1*. Caso contrário, terão 0 nesta entrega. É importante garantir que o ambiente de execução possa ser reproduzido pela equipa docente sem erros de compilação. Para tal, recomenda-se executar o projecto em cada um dos computadores dos membros da equipa. O projecto **obrigatoriamente** deverá correr num ambiente Docker. Falhas em **EXECUTAR** o projecto, num ambiente Docker, pela equipa docente, resultarão em **grave penalização nesta entrega**. Casos de uso que não sejam apresentados ou que apresentem falhas de execução, serão considerados como **não implementados**. Casos de uso parcialmente apresentados terão uma nota também parcial.

6 Critérios de Avaliação

Esta entrega corresponde a 35% da nota do projecto. Os restantes 65% serão avaliados na segunda fase e terceira fase. Para perceberem onde se devem focar, segue uma lista de critérios que serão avaliados:

6.1 Processo de Desenvolvimento e Demonstração

- Granularidade e frequência dos *commits*. Todos os membros do grupo devem contribuir de forma equitativa. *Disparidades significativas serão penalizadas*.
- Qualidade das mensagens de *commit*.
- Clareza na demonstração, funcionamento real da API REST apresentada e capacidade de explicação do trabalho desenvolvido pelo grupo nos vídeos.
- Quantidade e qualidade das regras de negócio implementadas e demonstradas nos vídeos.

6.2 Requisitos

- Em que medida os requisitos funcionais e não funcionais são abrangidos pela modelação.
- Implementação da REST API com os endpoints adequados e boas práticas (status codes, verbos HTTP).

7 Dicas

O projeto da disciplina de CSS é complexo, com muitas entidades, relacionamentos e casos de uso. Para garantir que conseguem entregar uma aplicação funcional, é importante que se organizem bem e sigam um processo iterativo de desenvolvimento. Algumas dicas para orientar o vosso trabalho:

- Comecem por discutir e desenhar o modelo de domínio, tendo em conta **todos** os casos de uso (os da Fase 1 e os futuros). **Não comecem a escrever código antes de terem um modelo bem definido.** Um modelo no papel é fácil de ajustar, enquanto reestruturar um projeto inteiro construído por cima de um modelo errado é uma enorme dor de cabeça.
- Não demorem muito tempo para começar o projeto. Quanto mais demoram para começar (e terem um modelo de domínio bem definido), mais difícil será corrigir erros posteriores e ter feedback dos professores.
- Utilizem as aulas práticas e os momentos com os docentes para validar a solução pensada. Idealmente, tirem dúvidas sobre o diagrama antes de começarem a programar, e voltem a pedir feedback depois de terem uma primeira versão da base de dados gerada.
- Criem uma pasta *docs* no repositório para partilhar os documentos gerados (e.g., diagramas, modelos, decisões de projeto) entre os membros do grupo e os professores.
- Ao começar a desenvolver, prestem atenção ao mapeamento das classes com as anotações JPA. Reflitam sobre as estratégias de herança, a cardinalidade dos relacionamentos, e a direção das associações (se faz sentido ser unidirecional ou bidirecional no vosso contexto).
- Identifiquem e discutam as garantias que o sistema tem de oferecer relativamente à lógica de negócio (e.g., um entregador não pode fazer duas entregas ao mesmo tempo; um pedido não avança para a cozinha sem estar pago).
- É fundamental garantir que o código escrito faz exatamente aquilo a que se propõe. Para isso, implementem testes unitários e de integração que validem se a lógica de negócio definida anteriormente está correta, incluindo os cenários de erro e exceções. Devem garantir uma cobertura de testes adequada (e.g., 80%), para os casos de uso implementados nesta fase.
- Preparem um roteiro que mostre o funcionamento dos casos de uso implementados. Certifiquem-se de que os vídeos estão bem organizados e que os endpoints são claramente apresentados (e.g., mostrando o JSON de requisição e resposta). Mantenham um documento de apoio para garantir que demonstraram todas as regras de negócio implementadas.

Nota Final: Em caso de dúvidas, utilizem o fórum de discussão no Moodle. Boa sorte com o desenvolvimento do *TascaEats*!